

Project: Course Study Buddy

Build a useful agent, one prompt at a time

You've read fourteen parts of the fundamentals guide. This project turns all of it into one program you'll actually use: an agent that quizzes you on this course, remembers which topics you keep getting wrong, and gets harder as you get better.

You build it by **prompting Claude Code**, milestone by milestone. Each milestone gives you one prompt, the things to check before moving on, and the failure that usually shows up. There is no solution code here on purpose — the point is that you can drive the SDK, not that you can copy it.

What you're building

By the end you'll have a `study-buddy` project that:

- loads your course notes from a local file and picks a topic to drill;
- asks a question, grades your answer, and explains what you missed;
- remembers your weak topics inside a context object the LLM never sees;
- raises or lowers difficulty on its own, per question;
- runs a question-writer and a grader as two specialist agents under one tutor;
- shows you the whole run as a trace you can read afterwards;
- hands a struggling student over to a remedial tutor that teaches instead of testing;
- keeps exam mode locked until the student has earned it, and never loops away your quota.

Before you start

You need the fundamentals project working — a `uv` project, a `.env` with `GEMINI_API_KEY`, and an agent that answers on `gemini-2.5-flash`. Parts 0–2 of the guide get you there. Every milestone below assumes the `llm_model` wiring from Part 2 is already in place.

Open a terminal in a **new** folder and run `claude`. Work in order.

How to work through this

Paste the prompt, then **read what comes back before you run it**. If a line surprises you, ask **why did you do it that way?** — that question is worth more than the milestone itself. Run the verify steps. Only then move on. A milestone that "works" but that you can't explain is a milestone you have to redo later, in front of the class.

Milestone 0 — Scaffold and smoke test

Get a running agent before you get a clever one. This is Parts 0–3 with nothing new.

```
Scaffold a uv-managed Python project called study-buddy with openai-agents
and python-dotenv. Wire an Agent to Gemini's gemini-2.5-flash through
AsyncOpenAI and OpenAIChatCompletionsModel, load GEMINI_API_KEY from .env,
gitignore .env, and give me an async main() run with asyncio.run() that
asks one hardcoded question. Keep it in main.py.
```

Verify

- `uv run main.py` prints an answer, not a traceback.
- `.env` is listed in `.gitignore`.
- The run goes through `Runner.run`, not `run_sync`. You'll need async later.

If it breaks: a 404 from the model almost always means the `base_url` lost its trailing `/v1beta/openai/`.

Milestone 1 — Notes the agent can read

An agent with no material to quiz you on is a chatbot. Give it your course notes as data, exposed through tools (Part 5).

```
Create topics.json holding the fourteen parts of my OpenAI Agents SDK
course.
Each entry needs: id, title, a three-sentence summary, and two key facts a
student must know. Then add two @function_tool functions – list_topics()
returning the ids and titles, and get_notes(topic_id: str) returning that
entry's summary and facts. Register both on the agent and prove they work
by asking it "what topics can you quiz me on?"
```

Verify

- The answer lists topics that exist in your file — not invented ones.
- `get_notes` is called for exactly the topic you name, and only that one.
- The docstrings read like instructions to the model, not notes to yourself.

If it breaks: if the agent invents topics, your tool descriptions are too vague — the model didn't realise it was supposed to look anything up.

Milestone 2 — A student profile the LLM never sees

Now the part that makes it yours. The agent needs to know who is studying and what they keep failing, without any of it leaking into the prompt (Part 7).

```
Add a StudentProfile dataclass with name, level ("beginner" or "intermediate"),
weak_topics as a list of ids, and answered as a dict of topic_id to correct/total
counts. Pass it to Runner.run via context=, and add a record_answer tool that
takes topic_id and was_correct, updates the profile through the
RunContextWrapper, and returns a short confirmation. Then print the tool's
params_json_schema and tell me which parameters the model can see.
```

Verify

- `record_answer`'s schema contains `topic_id` and `was_correct`, and **not** the wrapper.
- Two questions in a row on the same topic update the same counter object.
- Nothing in your prompt text mentions the student's name — the tools read it instead.

If it breaks: `AttributeError: 'NoneType' object has no attribute 'name'` means you forgot `context=profile` on that particular `Runner.run` call.

Milestone 3 — Difficulty that moves

A fixed system prompt can't teach. Build the prompt per turn from the profile (Part 8).

```
Replace the tutor's instructions string with a function taking (ctx, agent)
that returns different instructions depending on ctx.context: name the student,
name the agent, push toward topics in weak_topics, and demand harder questions
once accuracy on a topic passes 70%. Show me the resolved system prompt for
three fake profiles — a fresh student, one with two weak topics, and one who
is scoring well — before any model call happens.
```

Verify

- Three fake profiles produce three visibly different prompts.
- The function takes exactly two parameters. One or three raises `TypeError`.
- Difficulty is decided in Python, not by asking the model to "be adaptive".

If it breaks: if every profile yields the same prompt, you're reading a field that doesn't exist — print `ctx.context` and look.

Milestone 4 — Two specialists under one tutor

Writing a good question and grading an answer fairly are different jobs with different instructions. Split them (Part 11).

```
Create two specialist agents: a question_writer that outputs exactly one
question on a given topic and nothing else, and a grader that receives the
question, the student's answer and the key facts, then returns a verdict plus
one sentence of feedback. Wrap both with as_tool and give them to the tutor
agent. The tutor keeps the conversation – it should call the writer, show me
the question, take my answer, call the grader, then call record_answer.
```

Verify

- The final text you see comes from the tutor, in one consistent voice.
- Each specialist's generated tool takes a single `input` string — check the schema.
- The grader never sees your profile, only the question, answer and facts.

If it breaks: if the tutor answers its own quiz question, its instructions aren't strict enough about who does what. Tighten them before reaching for code.

Milestone 5 — Settings that match the job

Same model, three different jobs, three different settings (Part 6).

```
Give the question_writer temperature 0.9 so questions vary between runs, the
grader temperature 0.1 with a max_tokens ceiling so verdicts stay short and
repeatable, and set tool_choice="required" on the tutor for the turn where it
must record an answer. Run the same topic three times and show me what changed.
```

Verify

- Three runs on one topic produce three different questions.
- The same answer, graded twice, gets the same verdict.
- You can say out loud why `tool_choice="required"` is right there and wrong everywhere else.

If it breaks: if graded verdicts still wobble, `max_tokens` is truncating the verdict rather than shortening it. Ask for brevity in the instructions instead.

Milestone 6 — Two tutors, one definition

You now have a tutor worth reusing. Make variants without duplicating it (Part 9).

```
Clone the tutor into a gentle variant and a strict variant that differ only in
instructions and temperature. Prove with is-comparisons which attributes the
clones share with the base and which they don't, then show me what happens to
both clones when I append a tool to the base agent's tools list.
```

Verify

- Both clones still run on Gemini without repeating the `model=` wiring.
- The shared-list demonstration actually surprises you. If it doesn't, re-read the output.
- Overriding `model_settings` on a clone drops the fields you didn't restate.

If it breaks: nothing breaks here. That's the danger — the shared tools list is a bug you only meet in week three.

Milestone 7 — Watch the whole run

You've been flying blind. Turn the lights on (Part 10).

```
Remove set_tracing_disabled, call set_tracing_export_api_key with my OpenAI
key, and wrap one full quiz cycle — pick topic, ask, grade, record — in a
single with trace("Study session") block. Then read the trace at
platform.openai.com/traces and tell me which step took longest and which model
call I could have avoided.
```

Verify

- One quiz cycle is one trace, not four.
- You can name every span in it and say which agent produced it.
- You found at least one wasted call. There is always one.

If it breaks: an empty dashboard usually means `OPENAI_API_KEY` is not set, skipping trace export scrolled past in your console.

Milestone 8 — Ship it

An agent that only you can run isn't finished.

```
Turn main.py into a CLI loop: pick a topic or let the tutor pick from my weak
topics, answer in the terminal, see the verdict, and quit with a session
summary showing accuracy per topic. Persist the StudentProfile to a JSON file
between sessions. Then write a README with setup steps, one real example run,
and a section on what the agent does badly.
```

Verify

- Quitting and restarting keeps your history.
 - A stranger can run it with your README and nothing else.
 - The "what it does badly" section is honest and specific. That section is the one people read.
-

Milestone 9 — Hand the struggling student over

Quizzing someone who keeps failing the same topic is just repeating the failure. When a student misses a topic twice, the tutor should stop quizzing and **transfer the conversation** to a tutor that teaches (Part 12).

```
Add a Remedial Tutor agent that explains a topic from first principles with a
worked example and no quizzing. Give the main tutor a handoff to it, and
instruct it to hand off when the profile shows two or more wrong answers on
the current topic. Print result.last_agent after each run, and show me the
HandoffCallItem and HandoffOutputItem inside result.new_items when it fires.
```

Verify

- A student who fails twice ends up talking to the Remedial Tutor, not the quizzer.
- `result.last_agent.name` changes — that's your proof, not the tone of the reply.
- The handoff tool the model sees is named after the agent. Rename the agent, and the name changes with it.

If it breaks: if the handoff never fires, the routing rule is buried in a long instruction block. Move it to its own line and state the threshold as a number.

Milestone 10 — Continue the lesson, then come back

A handoff isn't one reply, it's a change of ownership. The remedial session should carry on with the specialist, and control should return deliberately (Part 12).

```
After a handoff, continue the next two turns with result.last_agent using
result.to_input_list() so the remedial tutor keeps context instead of
restarting. When the student answers a check question correctly, route back
to the main tutor with a fresh quiz on the same topic. Show me the agent name
that owned each of the four turns.
```

Verify

- The remedial tutor remembers what it just explained — no reintroductions.
- Turn ownership reads: tutor, remedial, remedial, tutor.
- Returning to the tutor is a decision in your code or instructions, not an accident.

If it breaks: if the specialist starts over each turn, you passed the original input instead of `to_input_list()`.

Milestone 11 — Gate exam mode, cap the loop

Two controls from Part 13. Exam mode should only exist for students who've earned it, and no quiz cycle should ever run away.

Add a `start_exam_mode` tool gated with `is_enabled` so it's only offered when the profile's level is "intermediate", and a `finish_session` tool that ends the run via `StopAtTools`. Wrap one full cycle in `try/except MaxTurnsExceeded` with a sensible `max_turns`. Then run the same question as a beginner and as an intermediate and show me which tools the model was offered each time.

Verify

- As a beginner, the model never mentions exam mode — it isn't refused, it's absent.
- `finish_session` ends the run, and its return value is the final output verbatim.
- You picked `max_turns` by counting the turns a real cycle needs, and can say the number out loud.

If it breaks: `MaxTurnsExceeded` on a normal cycle means you counted tool calls instead of LLM calls. A tool call plus the model reading its result is two turns.

Definition of done

Requirement	How it's checked
Runs on Gemini, keys in <code>.env</code> , <code>.env</code> gitignored	<code>uv run main.py</code> on a clean clone
Notes come from <code>topics.json</code> through tools	Agent can't quiz on a topic you deleted
<code>StudentProfile</code> passed as <code>context=</code> , never in the prompt	Tool schema shows no wrapper parameter
Instructions built per turn from the profile	Three profiles, three prompts
Writer and grader wrapped with <code>as_tool</code>	Final voice is the tutor's
Settings differ per specialist, with a reason	You can defend each number
One quiz cycle is one trace	Screenshot of the trace timeline
Profile survives a restart	Quit, rerun, history intact
Two wrong answers trigger a handoff	<code>result.last_agent.name</code> changes
The remedial session keeps its context	Four turns, no reintroductions
Exam mode is gated by level	Beginner run never sees the tool
The loop cannot run away	<code>MaxTurnsExceeded</code> caught, not crashed

What to hand in

Push the repo, then write three paragraphs in the README: what you built, one thing that broke and how you found it, and one thing you'd do differently. Include the trace screenshot. The debugging paragraph is the one that shows you learned something — a project with no failure story reads like a project someone else wrote.

If you want to push further

- Add a `Runner.run_streamed` mode so questions appear as they're typed.
- Give the grader a structured output instead of prose, and branch on it in Python.
- Let two tutor clones quiz you on the same topic and compare which teaches better.
- Give the handoff an `input_type` so the tutor must state *why* it's transferring.
- Replace `topics.json` with your own notes from another course and see what breaks.